



ELECTRÓNICA DIGITAL III

CLASE 5 Módulo Timer

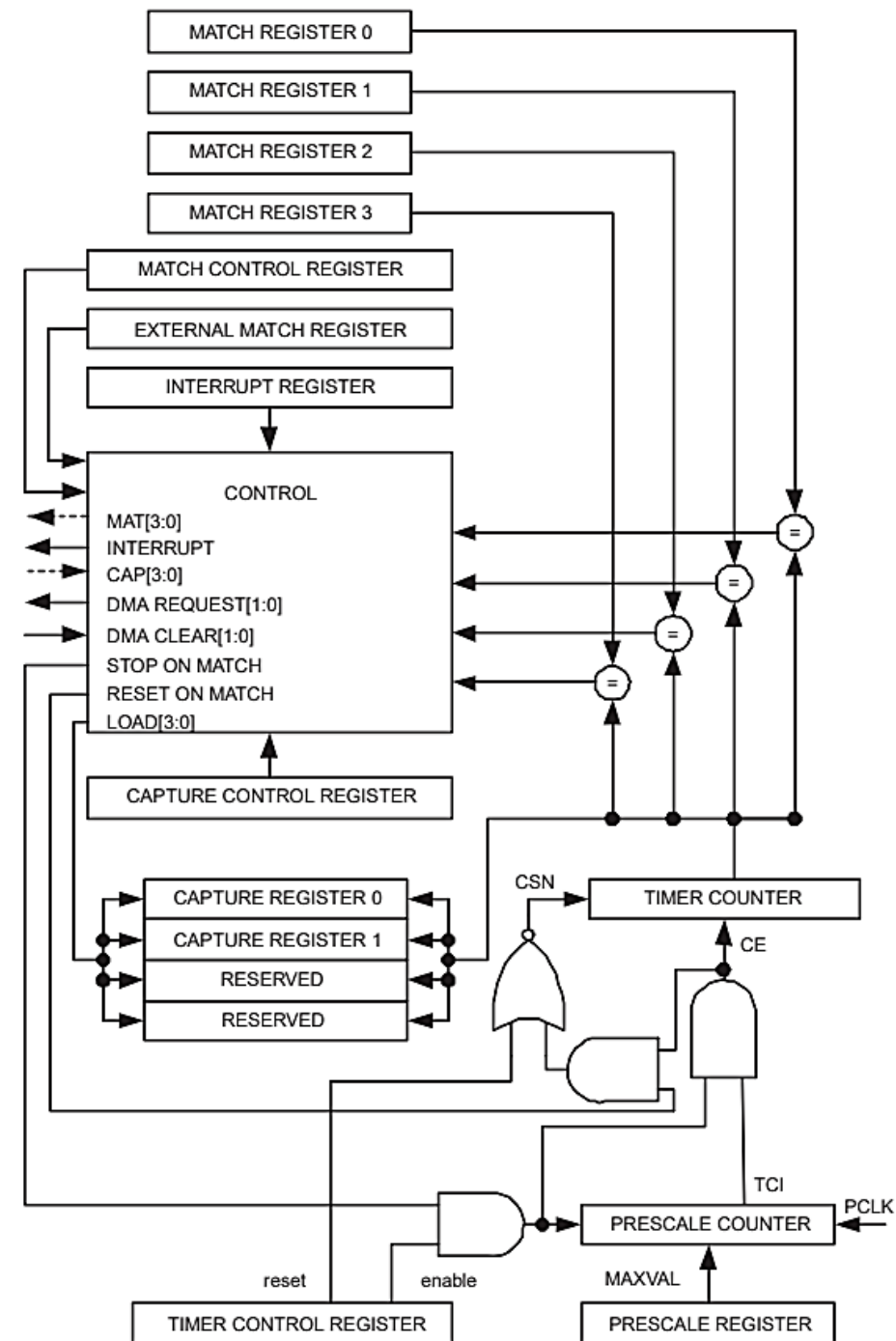
Ing. Emiliano Migliore

Módulo Timer

Los timers de 32bits (Timer0/1/2/3) de la LPC1769 pueden ser utilizados en **modo temporizador (MATCH)** o en **modo contador de tiempo entre eventos (CAPTURE)**.

Aplicaciones típicas

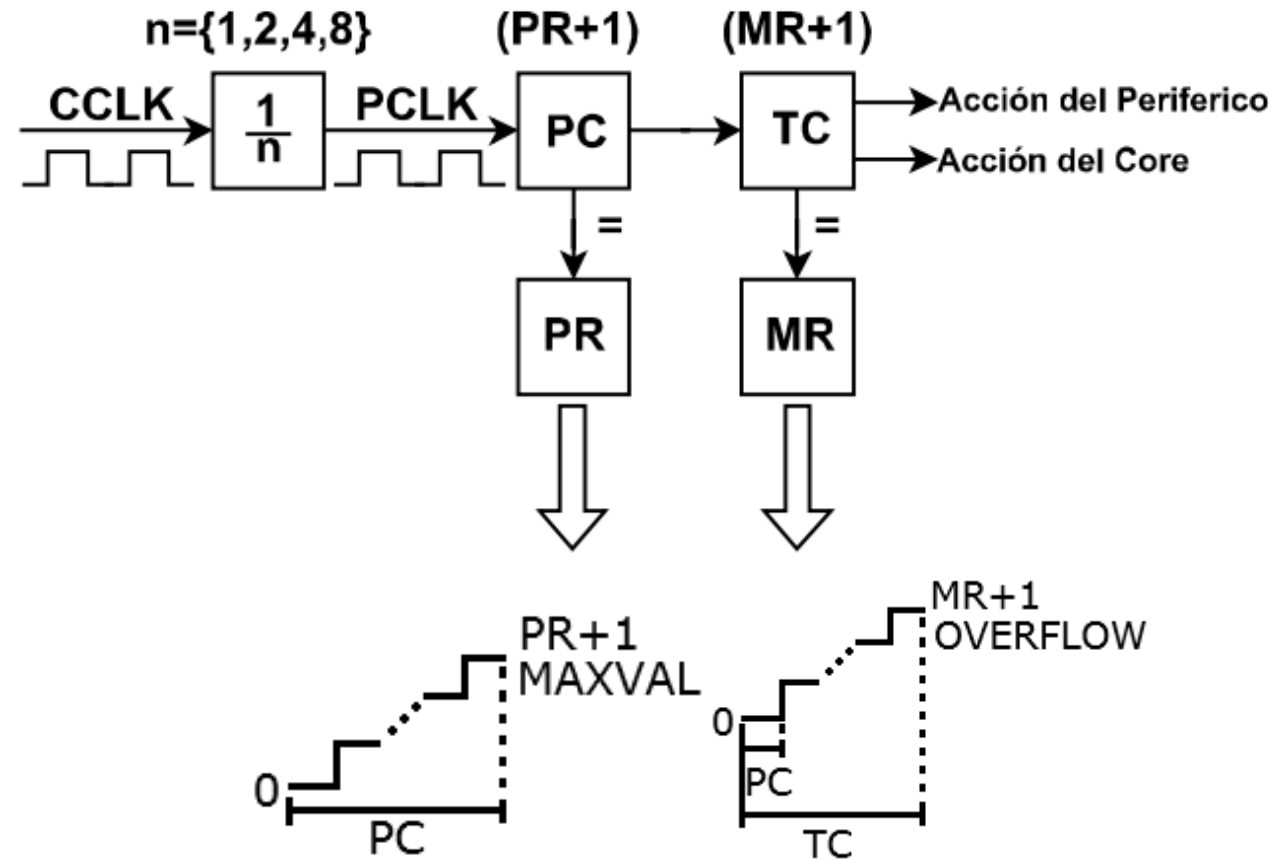
- Generación de señales periódicas o pulsos.
- Medición de frecuencias y períodos.
- Implementación de relojes en tiempo real de alta precisión (cuando no se requiere RTC).
- Captura de eventos rápidos en sistemas embebidos.
- Sincronización de procesos en aplicaciones de control.



Módulo Timer: Modo MATCH

Inicialmente, la señal de reloj CCLK ingresa a un divisor de frecuencia externo (PCLKSEL0) entregando la señal de reloj PCLK al periférico. Esta señal comanda al registro **PC (Prescale Counter)** que se compara con el valor del registro **PR (Prescale Register)**, y cuando se igualan, se reinicia a cero el registro PC y se incrementa en uno el registro **TC (Timer Counter)**. Por tanto, se produce una cuenta ascendente. A su vez, el registro TC se compara con los registros **MR[0/1/2/3] (Match Register 0/1/2/3)**, y cuando se igualan se produce alguno de los siguientes eventos de Match:

- **Acción del Periférico:** Generar señal de salida en un pin mediante el registro EMR (External Match Register)
 - Pin HIGH
 - Pin LOW
 - Pin TOGGLE (HIGH & LOW)
 - Pin NOTHING
- **Acción del Core:** Generar Interrupción mediante el NVIC



Módulo Timer: Modo MATCH

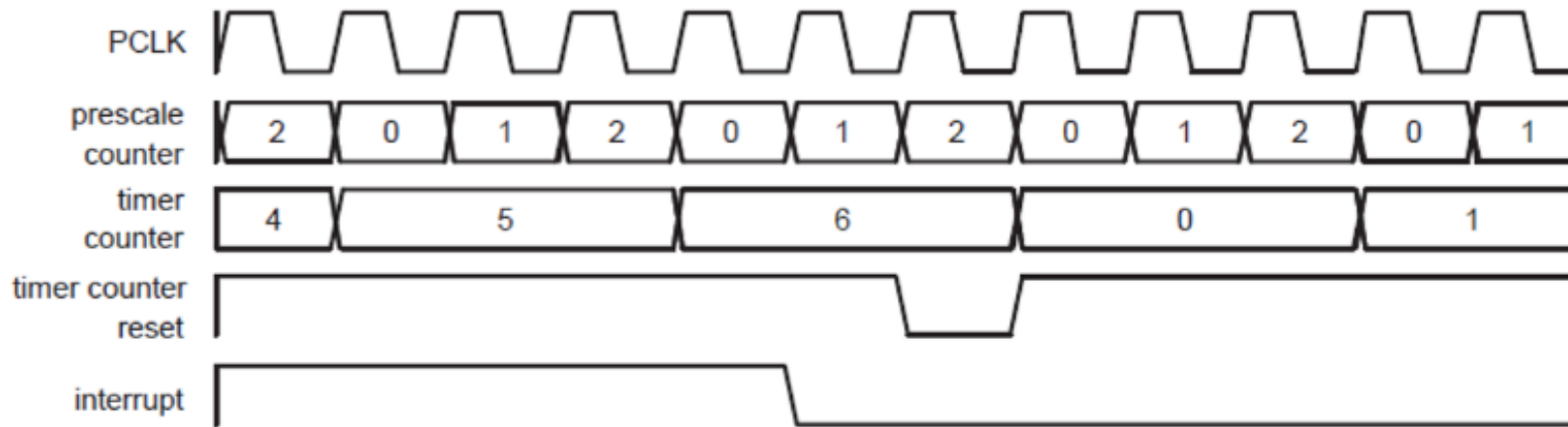


Fig 115. A timer cycle in which PR=2, MRx=6, and both interrupt and reset on match are enabled.

El valor de referencia para el MATCHx se define como:

$$t_{match} = \frac{1}{\left(\frac{CCLK}{n}\right)} (PR + 1)(MR + 1) \Rightarrow MR = \left\lceil t_{match} \left(\frac{CCLK}{n}\right) \frac{1}{(PR + 1)} \right\rceil - 1$$

Donde n es el valor del divisor de frecuencia externo dado por PCLKSEL0.

Módulo Timer: Inicialización del Timer

1) Se debe energizar el periférico a partir de los bits del registro PCON:

Power Control for Peripherals register (PCONP - address 0x400F C0C4) bit description

Bit	Symbol	Description	Reset value
1	PCTIM0	Timer/Counter 0 power/clock control bit.	1
2	PCTIM1	Timer/Counter 1 power/clock control bit.	1
22	PCTIM2	Timer 2 power/clock control bit.	0
23	PCTIM3	Timer 3 power/clock control bit.	0

Remark: On reset, Timer0/1 are enabled (PCTIM0/1 = 1), and Timer2/3 are disabled (PCTIM2/3 = 0).

Módulo Timer: Inicialización del Timer

2) Se debe seleccionar si la señal de reloj del periférico tendrá una división de frecuencia:

Peripheral Clock Selection register 0 (PCLKSEL0 - address 0x400F C1A8) bit description

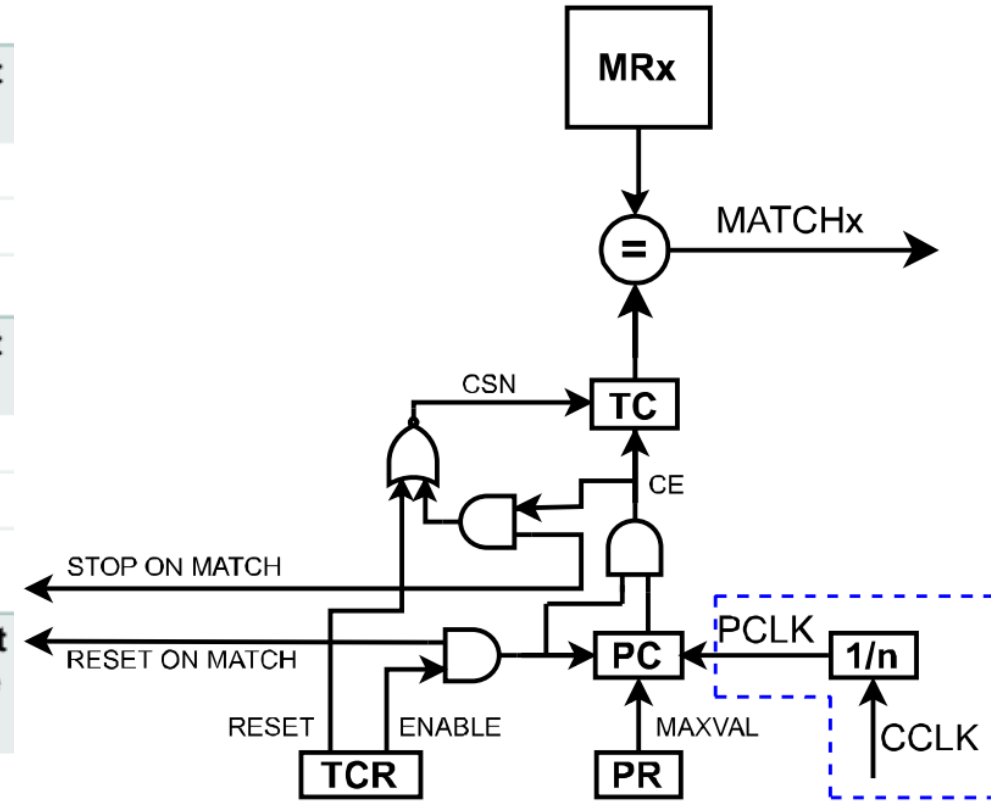
Bit	Symbol	Description	Reset value
3:2	PCLK_TIMER0	Peripheral clock selection for TIMER0.	00
5:4	PCLK_TIMER1	Peripheral clock selection for TIMER1.	00

Peripheral Clock Selection register 1 (PCLKSEL1 - address 0x400F C1AC) bit description

Bit	Symbol	Description	Reset value
13:12	PCLK_TIMER2	Peripheral clock selection for TIMER2.	00
15:14	PCLK_TIMER3	Peripheral clock selection for TIMER3.	00

Peripheral Clock Selection register bit values

PCLKSEL0 and PCLKSEL1 individual peripheral's clock select options	Function	Reset value
00	PCLK_peripheral = CCLK/4	00
01	PCLK_peripheral = CCLK	
10	PCLK_peripheral = CCLK/2	
11	PCLK_peripheral = CCLK/8, except for CAN1, CAN2, and CAN filtering when "11" selects = CCLK/6.	



Módulo Timer: Inicialización del Timer

3) Se debe seleccionar la modalidad de temporizador o contador del periférico a partir del registro CTCR:

Count Control Register (T[0/1/2/3]CTCR - addresses 0x4000 4070, 0x4000 8070, 0x4009 0070, 0x4009 4070) bit description

Bit	Symbol	Value	Description	Reset Value
1:0	Counter/ Timer Mode		This field selects which rising PCLK edges can increment the Timer's Prescale Counter (PC), or clear the PC and increment the Timer Counter (TC).	00
		00	Timer Mode: the TC is incremented when the Prescale Counter matches the Prescale Register. The Prescale Counter is incremented on every rising PCLK edge.	
		01	Counter Mode: TC is incremented on rising edges on the CAP input selected by bits 3:2.	
		10	Counter Mode: TC is incremented on falling edges on the CAP input selected by bits 3:2.	
		11	Counter Mode: TC is incremented on both edges on the CAP input selected by bits 3:2.	
3:2	Count Input Select		When bits 1:0 in this register are not 00, these bits select which CAP pin is sampled for clocking.	00
		00	CAPn.0 for TIMERN	
		01	CAPn.1 for TIMERN	
		10	Reserved	
		11	Reserved	
Note: If Counter mode is selected for a particular CAPn input in the TnCTCR, the 3 bits for that input in the Capture Control Register (TnCCR) must be programmed as 000. However, capture and/or interrupt can be selected for the other 3 CAPn inputs in the same timer.				
31:4	-	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

Módulo Timer: Inicialización del Timer

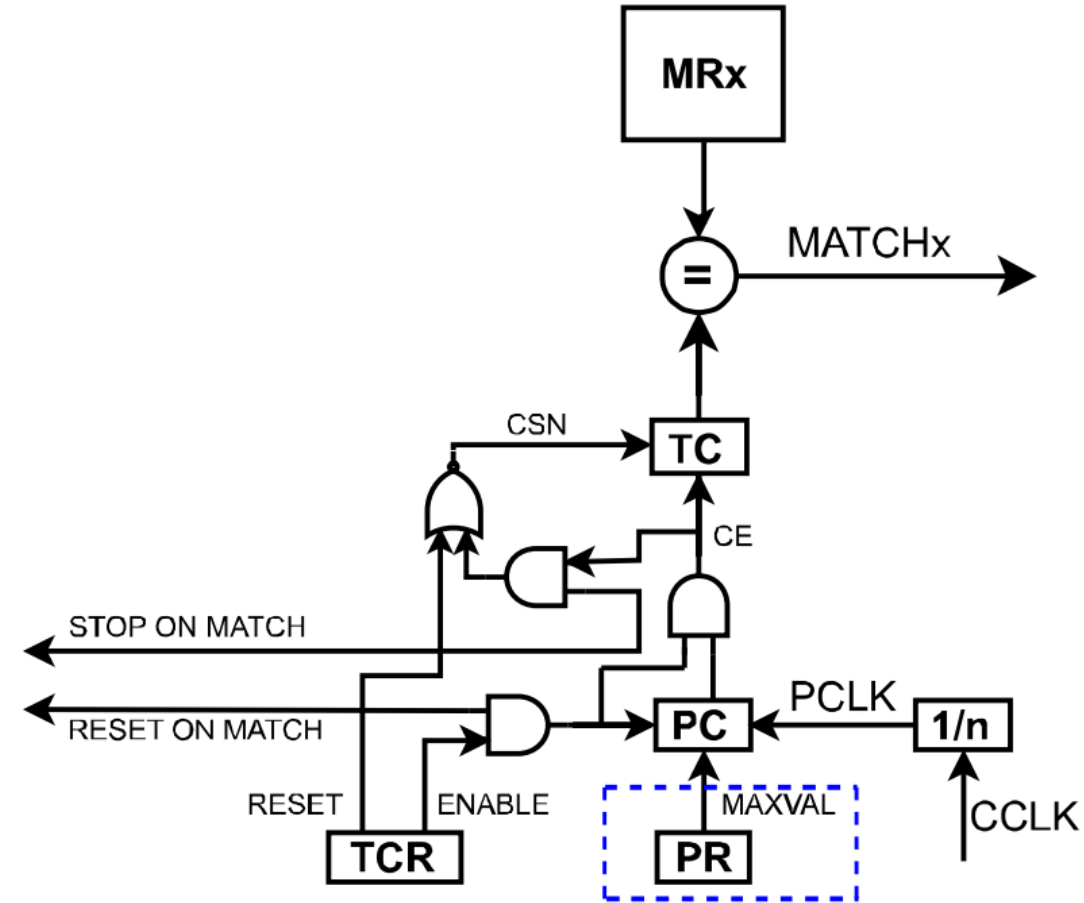
4) Se debe configurar el valor máximo que tendrá el registro PR:

Prescale register (T0PR - T3PR, 0x4000 400C, 0x4000 800C, 0x4009 000C, 0x4009 400C)

The 32-bit Prescale register specifies the maximum value for the Prescale Counter.

$$PR = PCLK_{TIMER0} (Time_{Overflow_{PC}} / 1 \times 10^6)$$

Donde $Time_{Overflow_{PC}}$ es la base de tiempo de desborde del Prescaler en $[\mu s]$.



Módulo Timer: Inicialización del Timer

5) Se debe reiniciar el periférico:

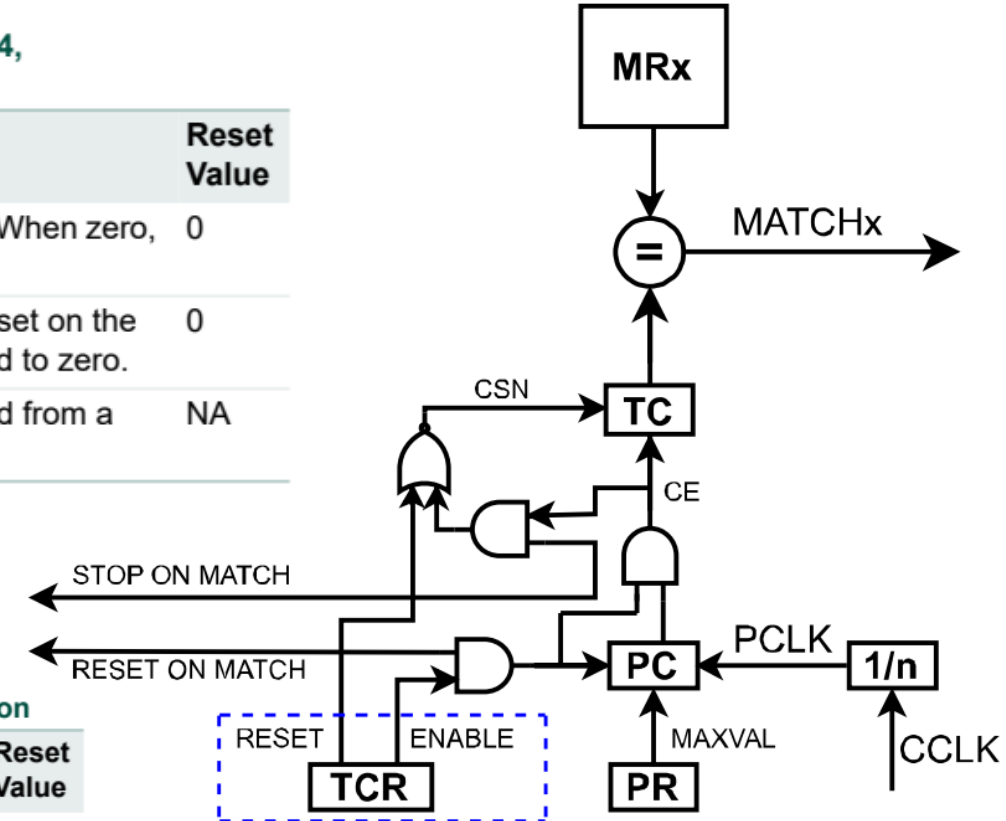
Timer Control Register (TCR, TIMERN: TnTCR - addresses 0x4000 4004, 0x4000 8004, 0x4009 0004, 0x4009 4004) bit description

Bit	Symbol	Description	Reset Value
0	Counter Enable	When one, the Timer Counter and Prescale Counter are enabled for counting. When zero, the counters are disabled.	0
1	Counter Reset	When one, the Timer Counter and the Prescale Counter are synchronously reset on the next positive edge of PCLK. The counters remain reset until TCR[1] is returned to zero.	0
31:2	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

6) Se debe limpiar la bandera de interrupción por MATCH (MRx) o por CAPTURE (CRx):

Interrupt Register (T[0/1/2/3]IR - addresses 0x4000 4000, 0x4000 8000, 0x4009 0000, 0x4009 4000) bit description

Bit	Symbol	Description	Reset Value
0	MR0 Interrupt	Interrupt flag for match channel 0.	0
1	MR1 Interrupt	Interrupt flag for match channel 1.	0
2	MR2 Interrupt	Interrupt flag for match channel 2.	0
3	MR3 Interrupt	Interrupt flag for match channel 3.	0
4	CR0 Interrupt	Interrupt flag for capture channel 0 event.	0
5	CR1 Interrupt	Interrupt flag for capture channel 1 event.	0
31:6	-	Reserved	-

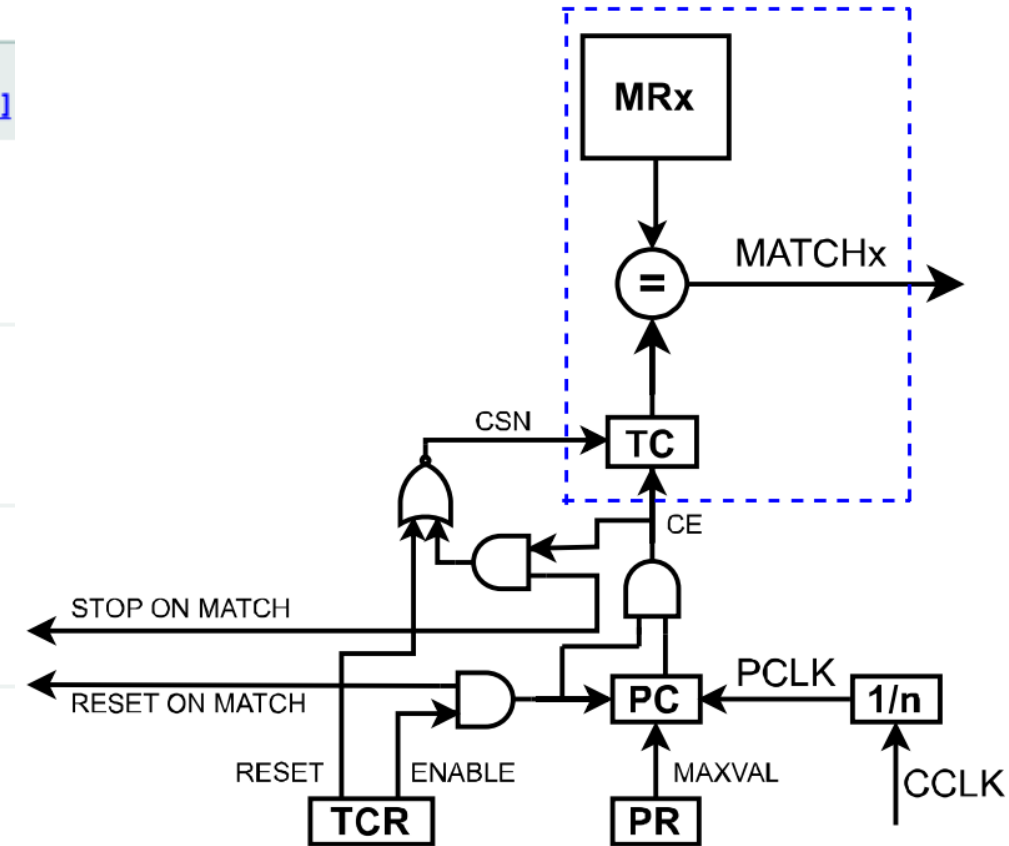


Módulo Timer: Configuración del Timer en Modo Match

1) Se debe establecer el valor máximo del registro TC a partir de los registros MRx:

TIMER/COUNTER0-3 register map

Generic Name	Description	Access	Reset Value ^[1]
MR0	Match Register 0. MR0 can be enabled through the MCR to reset the TC, stop both the TC and PC, and/or generate an interrupt every time MR0 matches the TC.	R/W	0
MR1	Match Register 1. See MR0 description.	R/W	0
MR2	Match Register 2. See MR0 description.	R/W	0
MR3	Match Register 3. See MR0 description.	R/W	0



Módulo Timer: Configuración del Timer en Modo Match

2) Se debe configurar la opción de interrupción por MATCH, la opción de habilitación de reset por MATCH, y la opción de stop por MATCH:

Match Control Register (T[0/1/2/3]MCR - addresses 0x4000 4014, 0x4000 8014, 0x4009 0014, 0x4009 4014) bit description

Bit	Symbol	Value	Description	Reset Value	Bit	Symbol	Value	Description	Reset Value
0	MR0I	1	Interrupt on MR0: an interrupt is generated when MR0 matches the value in the TC.	0	6	MR2I	1	Interrupt on MR2: an interrupt is generated when MR2 matches the value in the TC.	0
		0	This interrupt is disabled				0	This interrupt is disabled	
1	MR0R	1	Reset on MR0: the TC will be reset if MR0 matches it.	0	7	MR2R	1	Reset on MR2: the TC will be reset if MR2 matches it.	0
		0	Feature disabled.				0	Feature disabled.	
2	MR0S	1	Stop on MR0: the TC and PC will be stopped and TCR[0] will be set to 0 if MR0 matches the TC.	0	8	MR2S	1	Stop on MR2: the TC and PC will be stopped and TCR[0] will be set to 0 if MR2 matches the TC.	0
		0	Feature disabled.				0	Feature disabled.	
3	MR1I	1	Interrupt on MR1: an interrupt is generated when MR1 matches the value in the TC.	0	9	MR3I	1	Interrupt on MR3: an interrupt is generated when MR3 matches the value in the TC.	0
		0	This interrupt is disabled				0	This interrupt is disabled	
4	MR1R	1	Reset on MR1: the TC will be reset if MR1 matches it.	0	10	MR3R	1	Reset on MR3: the TC will be reset if MR3 matches it.	0
		0	Feature disabled.				0	Feature disabled.	
5	MR1S	1	Stop on MR1: the TC and PC will be stopped and TCR[0] will be set to 0 if MR1 matches the TC.	0	11	MR3S	1	Stop on MR3: the TC and PC will be stopped and TCR[0] will be set to 0 if MR3 matches the TC.	0
		0	Feature disabled.				0	Feature disabled.	
					31:12	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.		NA

Módulo Timer: Configuración del Timer en Modo Match

3) Se debe configurar el evento a producirse luego de que ocurra una interrupción por MATCH a partir del registro EMR:

External Match Register (T[0/1/2/3]EMR - addresses 0x4000 403C, 0x4000 803C, 0x4009 003C, 0x4009 403C) bit description

Bit	Symbol	Description	Reset Value
0	EM0	External Match 0. When a match occurs between the TC and MR0, this bit can either toggle, go low, go high, or do nothing, depending on bits 5:4 of this register. This bit can be driven onto a MATn.0 pin, in a positive-logic manner (0 = low, 1 = high).	0
1	EM1	External Match 1. When a match occurs between the TC and MR1, this bit can either toggle, go low, go high, or do nothing, depending on bits 7:6 of this register. This bit can be driven onto a MATn.1 pin, in a positive-logic manner (0 = low, 1 = high).	0
2	EM2	External Match 2. When a match occurs between the TC and MR2, this bit can either toggle, go low, go high, or do nothing, depending on bits 9:8 of this register. This bit can be driven onto a MATn.2 pin, in a positive-logic manner (0 = low, 1 = high).	0
3	EM3	External Match 3. When a match occurs between the TC and MR3, this bit can either toggle, go low, go high, or do nothing, depending on bits 11:10 of this register. This bit can be driven onto a MATn.3 pin, in a positive-logic manner (0 = low, 1 = high).	0
5:4	EMC0	External Match Control 0. Determines the functionality of External Match 0. Table 433 shows the encoding of these bits.	00
7:6	EMC1	External Match Control 1. Determines the functionality of External Match 1. Table 433 shows the encoding of these bits.	00
9:8	EMC2	External Match Control 2. Determines the functionality of External Match 2. Table 433 shows the encoding of these bits.	00
11:10	EMC3	External Match Control 3. Determines the functionality of External Match 3. Table 433 shows the encoding of these bits.	00
15:12	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

External Match Control

EMR[11:10], EMR[9:8], EMR[7:6], or EMR[5:4]	Function
00	Do Nothing.
01	Clear the corresponding External Match bit/output to 0 (MATn.m pin is LOW if pinned out).
10	Set the corresponding External Match bit/output to 1 (MATn.m pin is HIGH if pinned out).
11	Toggle the corresponding External Match bit/output.

Ejemplo 1: Interrupción por MATCH del TMR0

```
1  /*
2   * @brief   Enciende y apaga un LED conectado al pin P0.22 de forma intermitente
3   *          cada 1s debido a la interrupción del TMR0
4   */
5
6  /* Directivas de Inclusión */
7  #include "LPC17xx.h"
8  #include "lpc17xx_gpio.h"
9  #include "lpc17xx_pinsel.h"
10 #include "lpc17xx_timer.h"
11
12 /* Definición de Constantes y Macros */
13 #define OUTPUT (uint8_t) 1
14 #define INPUT  (uint8_t) 0
15 #define PIN_22 ((uint32_t)(1<<22))
16 #define PORT_0 (uint8_t) 0
17
18 /* Definición de Variables Globales */
19
20 /* Prototipos de Funciones */
21 void cfgGPIO(void);
22 void cfgTimer(void);
23
24 /* Función Principal */
25 int main(void) {
26
27     cfgGPIO();
28     cfgTimer();
29
30     while(1) {}
31
32     return 0;
33 }
```

Ejemplo 1: Interrupción por MATCH del TMR0

```
34
35  /* Definiciones de Funciones */
36  void cfgGPIO(void)
37  {
38      PINSEL_CFG_Type cfgPinLED;           //Configuración del PIN P0_22
39      PINSEL_CFG_Type confPinMAT0;         //Configuración del PIN P1_28
40
41      cfgPinLED.Portnum = PINSEL_PORT_0;   //Selección de Puerto 0
42      cfgPinLED.Pinnum = PINSEL_PIN_22;    //Selección de Pin 22
43      cfgPinLED.Funcnum = PINSEL_FUNC_0;   //Funcionalidad de P0.22 como GPIO
44      cfgPinLED.Pinmode = PINSEL_PINMODE_TRISTATE; //Pin Flotante
45      cfgPinLED.OpenDrain = PINSEL_PINMODE_NORMAL; //Modalidad sin Drenador Abierto
46      //LPC_PINCON->PINSEL1 &= ~(3<<12); //Funcionalidad de P0.22 como GPIO
47      GPIO_SetDir(PORT_0, PIN_22, OUTPUT); //P0.22 Output
48      //LPC_GPIO0->FIODIR |= (1<<22);     //P0.22 Output
49
50      confPinMAT0.Portnum = PINSEL_PORT_1; //Selección de Puerto 1
51      confPinMAT0.Pinnum = PINSEL_PIN_28;  //Selección de Pin 28
52      confPinMAT0.Funcnum = PINSEL_FUNC_3; //Funcionalidad de P1.28 como MAT0.0
53      confPinMAT0.OpenDrain = PINSEL_PINMODE_NORMAL; //Modalidad sin Drenador Abierto
54
55      PINSEL_ConfigPin(&cfgPinLED);         //Iniciación LED: P0.22 (Output)
56      PINSEL_ConfigPin(&confPinMAT0);      //Iniciación TMR0: P1.28 (MAT0.0)
57      return;
58  }
```

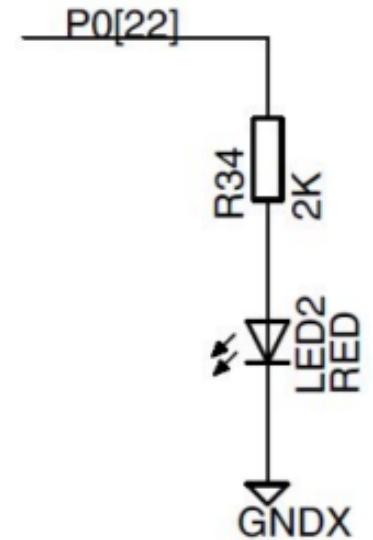
Ejemplo 1: Interrupción por MATCH del TMR0

```
59
60 void cfgTimer(void)
61 {
62     TIM_TIMERCFG_Type    cfgTimerMode;           //Configuración de Timer en Modo Temporizador
63     TIM_MATCHCFG_Type    cfgTimerMatch;          //Configuración del Tiempo de Desborde de MATCH
64
65     cfgTimerMode.PrescaleOption = TIM_PRESCALE_USVAL; //Base de Tiempo del Prescaler a partir de valor en µs
66     cfgTimerMode.PrescaleValue  = 1000;           //Base de Tiempo de Desborde de Prescaler [µs] (Time_Overflow_PC)
67     //LPC_SC->PCONP |= (1<<1);                   //Encendido del Periférico (TMR0 ON) - [Default: ON]
68     //LPC_SC->PCLKSEL0 &= ~(0x03<<2);           //PCLK_TIMER0 = CCLK/4
69     //LPC_TIM0->CTCR &= ~(3<<0);                 //Timer Mode (Incremento de TC por Match de PR) - [Default: 00]
70     //LPC_TIM0->PR = 25e6*(1000/1e6);             //Valor Máximo para PC
71     //LPC_TIM0->TCR |= (1<<1);                   //Habilitación Reset del TIMER0: (Counter Reset=1)
72     //LPC_TIM0->TCR &= ~(1<<1);                 //Deshabilitación Reset del TIMER0: (Counter Reset=0)
73     //LPC_TIM0->IR |= (1<<0);                    //Limpia Bandera de Interrupción por MATCH0
74
75     cfgTimerMatch.MatchChannel = 0;               //Habilitación de Canal de MATCH0: (MR0)
76     cfgTimerMatch.MatchValue   = 999;            //Valor Máximo para TC (MR)
77     cfgTimerMatch.IntOnMatch   = ENABLE;          //Habilitación de Interrupción por MATCH: (MR0I=1)
78     cfgTimerMatch.ResetOnMatch = ENABLE;          //Habilitación de Reset por MATCH: (MR0R=1)
79     cfgTimerMatch.StopOnMatch  = DISABLE;         //Deshabilitación de Stop por MATCH: (MR0S=0)
80     cfgTimerMatch.ExtMatchOutputType = TIM_EXTMATCH_NOHING; //Modalidad NINGUNA en P1.28
81     //LPC_TIM0->MR0 = 999;                        //Valor Máximo para TC
82     //LPC_TIM0->MCR |= (1<<0);                    //Habilitación de Interrupción por MATCH: (MR0I=1)
83     //LPC_TIM0->MCR |= (1<<1);                    //Habilitación de Reset por MATCH: (MR0R=1)
84     //LPC_TIM0->MCR &= ~(1<<2);                  //Deshabilitación de Stop por MATCH: (MR0S=0)
85     //LPC_TIM0->EMR &= ~(0x03<<4);               //Modalidad Ninguna en P1.28: (EMC0=00)
86
87     TIM_Init(LPC_TIM0, TIM_TIMER_MODE, &cfgTimerMode); //Configuración del Modalidad del TIMER0
88     TIM_ConfigMatch(LPC_TIM0, &cfgTimerMatch);          //Configuración del TIMER0 como Temporizador
89     TIM_Cmd(LPC_TIM0, ENABLE);                          //Habilitación del TIMER0
90     //LPC_TIM0->TCR |= (1<<0);                    //Habilitación del TIMER0: (Counter Enable=1)
91
92     NVIC_EnableIRQ(TIMER0_IRQn);                      //Habilitación de Interrupción por TIMER0
93     return;
94 }
```

$$\begin{aligned} t_{\text{delay}} &= \overset{\text{nothing}}{\hat{1}} t_{\text{MATCH}_{\text{TMR0}}} \Rightarrow 1\text{s} \\ MR &= \frac{t_{\text{MATCH}_{\text{TMR0}}}}{t_{\text{OVERFLOW}_{\text{PC}}}} - 1 \\ &= \frac{\frac{t_{\text{delay}}}{1}}{t_{\text{OVERFLOW}_{\text{PC}}}} - 1 \\ &= \frac{\frac{1\text{s}}{1}}{1000\mu\text{s}} - 1 \\ &= 999 \end{aligned}$$

Ejemplo 1: Interrupción por MATCH del TMR0

```
94
95 void TIMER0_IRQHandler(void)
96 {
97     if(TIM_GetIntStatus(LPC_TIM0, TIM_MR0_INT)){           //Consulta Interrupción por MR0 (MATCH0 del Timer0)
98         //if(LPC_TIM0->IR & (1<<0)){
99         //Ejecución de la ISR
100         static uint8_t i = 0;
101         if(i==0){
102             GPIO_SetValue(PORT_0,PIN_22);
103             //LPC_GPIO0->FIOSET = PIN_22;
104             i = 1;
105         }else if(i==1){
106             GPIO_ClearValue(PORT_0,PIN_22);
107             //LPC_GPIO0->FIOCLR = PIN_22;
108             i = 0;
109         }
110         //}
111     }
112
113     TIM_ClearIntPending(LPC_TIM0, TIM_MR0_INT);           //Limpia Bandera de Interrupción por TIMER0
114     //LPC_TIM0->IR |= (1<<0);
115     return;
116 }
117
```



Ejercicio 1: Interrupción por Timer

Calcular cual es el tiempo máximo que se puede temporizar utilizando un timer en modo match con máximo valor de prescaler y máximo divisor de frecuencia de periférico. Especificar el valor a cargar en los correspondientes registros del timer. Suponer una frecuencia de core cclk de 50 MHz.

Ejercicio 2: Interrupción por Timer

Escribir el código que configure el timer0 para cumplir con las especificaciones dadas en la figura adjunta. (Pag 510 Figura 115 del manual de usuario del LPC1769). Considerar una frecuencia de cclk de 100 Mhz y una división de reloj de periférico de 2.

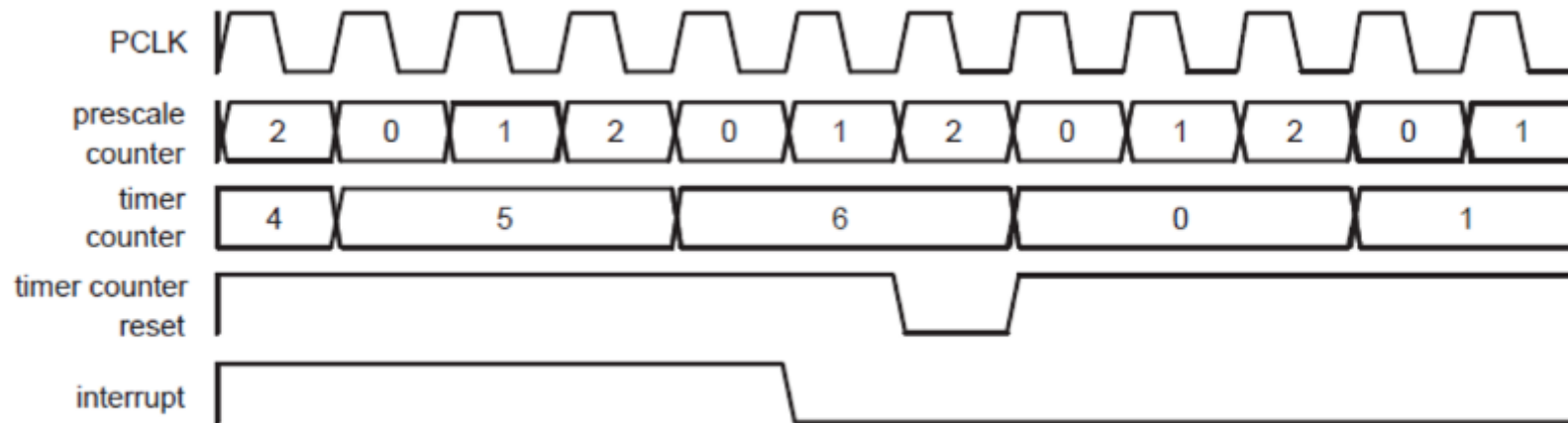


Fig 115. A timer cycle in which PR=2, MRx=6, and both interrupt and reset on match are enabled.

Ejercicio 3: Interrupción por Timer

Escribir un programa para que por cada presión de un pulsador, la frecuencia de parpadeo de un led disminuya a la mitad debido a la modificación del prescaler del Timer 2. El pulsador debe producir una interrupción por EINT1 con flanco descendente.

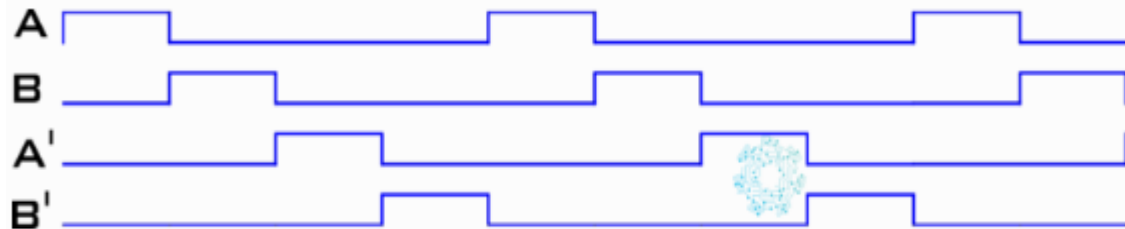
Ejercicio 4: Interrupción por Timer

Escribir un programa para que por cada presión de un pulsador, la frecuencia de parpadeo disminuya a la mitad debido a la modificación del registro del Match 0. El pulsador debe producir una interrupción por EINT2 con flanco descendente.

Ejercicio 5: Interrupción por Timer

Utilizando los 4 registros match del Timer 0 y las salidas P0.0, P0.1, P0.2 y P0.3, realizar un programa en C que permita obtener las formas de ondas adjuntas, donde los pulsos en alto tienen una duración de 5 ms. Un pulsador conectado a la entrada EINT3, permitirá elegir entre las dos secuencias mediante una rutina de servicio a la interrupción. La prioridad de la interrupción del Timer tiene que ser mayor que la del pulsador. Estas formas de ondas son muy útiles para controlar un motor paso a paso.

(Par menor, consumo bajo)



(Par máximo, buena velocidad, alto consumo)



Módulo Timer: Modo CAPTURE

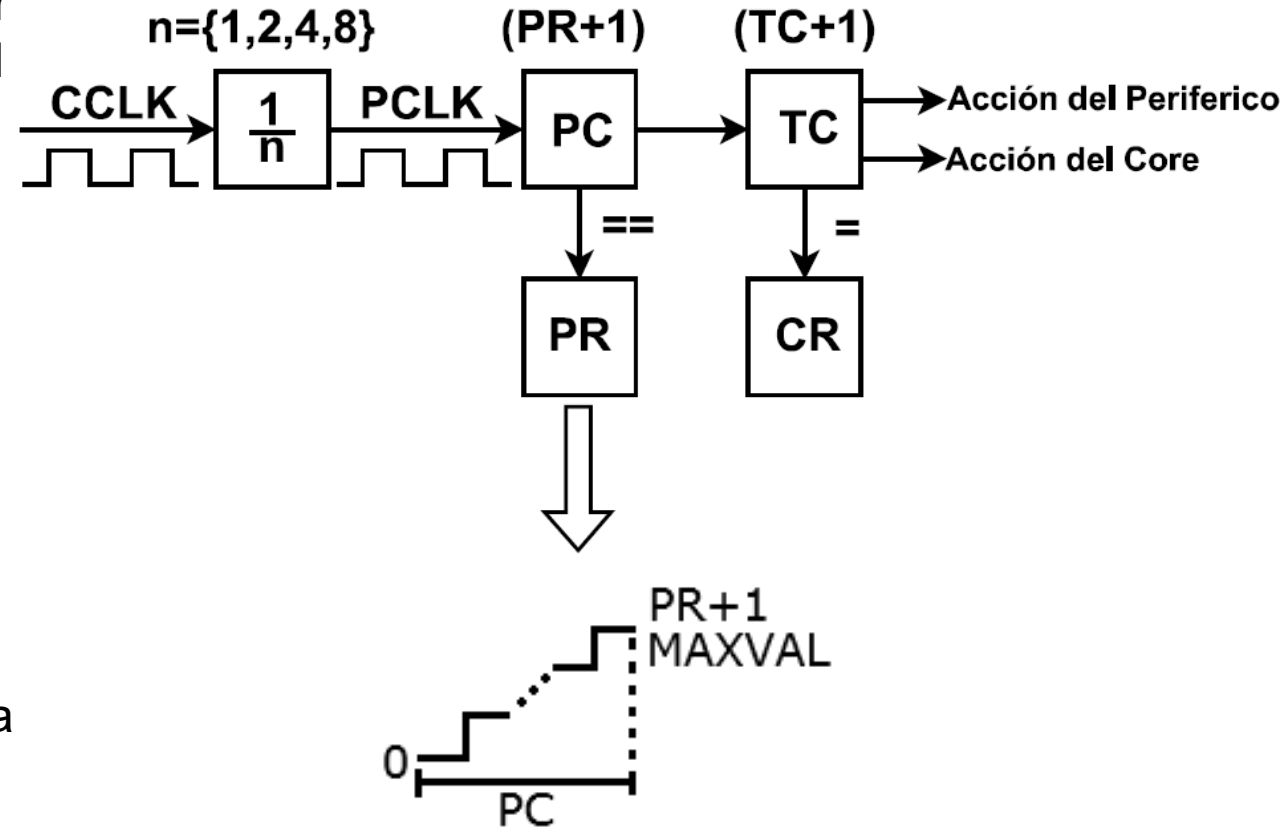
Inicialmente, la señal de reloj CCLK ingresa a un divisor de frecuencia externo (PCLKSEL0) entregando la señal de reloj PCLK al periférico. Esta señal comanda al registro **PC (Prescale Counter)** que se compara con el valor del registro **PR (Prescale Register)** y cuando se igualan, se reinicia a cero el registro PC y se incrementa en uno el registro **TC (Timer Counter)**, por tanto, se produce una cuenta ascendente (1 tick). Luego, cuando se produce un evento en los pines de entrada CAP[3:0], el registro TC se guarda en los registros **CR[0/1] (Capture Register 0/1)**.

El tiempo correspondiente a un tick del TC se determina como:

$$t_{capture} = \frac{1}{\left(\frac{CCLK}{n}\right)} (PR + 1)$$

Entonces, el tiempo entre un evento de captura será:

$$t_{delay} = t_{capture} CR$$



Nota:

Match: evento generado por comparación *interna* ($TC == MR$).

Capture: evento generado por *señal externa*, que guarda el valor de **TC**.

Ejemplo 2: Interrupción por CAPTURE del TMR0

```
1  /*
2  * @brief  Enciende un LED conectado al pin P0.22 cuando el tiempo entre dos eventos consecutivos
3  *         asociados a un flanco ascendente en el pin P1.26 debido a la interrupción del TMR0, sea
4  *         mayor o igual a 1 segundo.
5  */
6
7  /* Directivas de Inclusión */
8  #include "LPC17xx.h"
9  #include "lpc17xx_gpio.h"
10 #include "lpc17xx_pinsel.h"
11 #include "lpc17xx_timer.h"
12
13 /* Definición de Constantes y Macros */
14 #define OUTPUT (uint8_t) 1
15 #define INPUT  (uint8_t) 0
16 #define PIN_22 ((uint32_t)(1<<22))
17 #define PORT_0 (uint8_t) 0
18
19 /* Definición de Variables Globales */
20 volatile uint32_t lastCapValue = 0;
21 volatile uint32_t currCapValue = 0;
22 volatile uint8_t capFlag = 0;
23
24 /* Prototipos de Funciones */
25 void cfgGPIO(void);
26 void cfgTimer(void);
27
28 /* Función Principal */
29 int main(void) {
30
31     cfgGPIO();
32     cfgTimer();
33
34     while(1){}
35
36     return 0;
37 }
38
```

Ejemplo 2: Interrupción por CAPTURE del TMR0

```
39  /* Definiciones de Funciones */
40  void cfgGPIO(void)
41  {
42      PINSEL_CFG_Type cfgPinLED;           //Configuración del PIN P0_22
43      PINSEL_CFG_Type cfgPinCAP0;         //Configuración del PIN P1_26
44
45      cfgPinLED.Portnum    = PINSEL_PORT_0; //Selección de Puerto 0
46      cfgPinLED.Pinnum     = PINSEL_PIN_22; //Selección de Pin 22
47      cfgPinLED.Funcnum    = PINSEL_FUNC_0; //Funcionalidad de P0.22 como GPIO
48      cfgPinLED.Pinmode    = PINSEL_PINMODE_TRISTATE; //Pin Flotante
49      cfgPinLED.OpenDrain  = PINSEL_PINMODE_NORMAL; //Modalidad sin Drenador Abierto
50      //LPC_PINCON->PINSEL1 &= ~(3<<12); //Funcionalidad de P0.22 como GPIO
51      GPIO_SetDir(PORT_0, PIN_22, OUTPUT); //P0.22 Output
52      //LPC_GPIO0->FIODIR |= (1<<22); //P0.22 Output
53
54      cfgPinCAP0.Portnum = PINSEL_PORT_1; //Selección de Puerto 1
55      cfgPinCAP0.Pinnum  = PINSEL_PIN_26; //Selección de Pin 26
56      cfgPinCAP0.Funcnum = PINSEL_FUNC_3; //Funcionalidad de P1.26 como CAP0.0
57      cfgPinCAP0.OpenDrain = PINSEL_PINMODE_NORMAL; //Modalidad sin Drenador Abierto
58
59      PINSEL_ConfigPin(&cfgPinLED); //Inicialización LED: P0.22 (Output)
60      PINSEL_ConfigPin(&cfgPinCAP0); //Inicialización TMR0: P1.26 (CAP0.0)
61      return;
62  }
63
```

Ejemplo 2: Interrupción por CAPTURE del TMR0

```
64 void cfgTimer(void)
65 {
66     TIM_TIMERCFG_Type    cfgTimerMode;           //Configuración de Timer en Modo Temporizador
67     TIM_MATCHCFG_Type    cfgTimerCapture;        //Configuración del Tiempo de Desborde de MATCH
68
69     cfgTimerMode.CountInputSelect = TIM_COUNTER_INCAP0; //Pin de entrada CAP0.0 para TIMER0
70     //LPC_SC->PCONP |= (1<<1);                  //Encendido del Periférico (TMR0 ON) - [Default: ON]
71     //LPC_SC->PCLKSEL0 &= ~(0x03<<2);           //PCLK_TIMER0 = CCLK/4
72     //LPC_TIM0->CTCR &= ~(3<<0);                //Timer Mode (Incremento de TC por Match de PR) - [Default: 00]
73     //LPC_TIM0->PR = 0;                          //Valor Máximo para PC
74     //LPC_TIM0->TCR |= (1<<1);                  //Habilitación Reset del TIMER0: (Counter Reset=1)
75     //LPC_TIM0->TCR &= ~(1<<1);                 //Deshabilitación Reset del TIMER0: (Counter Reset=0)
76     //LPC_TIM0->IR |= (1<<4);                   //Limpia Bandera de Interrupción por CAPTURE0
77
78     cfgTimerCapture.CaptureChannel = 0;          //Habilitación de Canal de CAPTURE0: (MR0)
79     cfgTimerCapture.RisingEdge     = ENABLE;     //Evento de CAPTURE por Flanco Ascendente (CAP0RE=1)
80     cfgTimerCapture.FallingEdge    = DISABLE;    //Evento de CAPTURE por Flanco Descendente (CAP0FE=0)
81     cfgTimerCapture.IntOnCaption    = ENABLE;     //Interrupción durante el Evento de CAPTURE (CAP0I=1)
82     //LPC_TIM0->CCR |= (1<<0);                  //Evento de CAPTURE por Flanco Ascendente (CAP0RE=1)
83     //LPC_TIM0->CCR &= ~(1<<1);                 //Evento de CAPTURE por Flanco Descendente (CAP0FE=0)
84     //LPC_TIM0->CCR |= (1<<2);                   //Interrupción durante el Evento de CAPTURE (CAP0I=1)
85
86     TIM_Init(LPC_TIM0, TIM_COUNTER_RISING_MODE, &cfgTimerMode); //Configuración del Modalidad del TIMER0
87     TIM_ConfigCapture(LPC_TIM0, &cfgTimerCapture); //Configuración del TIMER0 como Contador
88     TIM_Cmd(LPC_TIM0, ENABLE); //Habilitación del TIMER0
89     //LPC_TIM0->TCR |= (1<<0);                  //Habilitación del TIMER0: (Counter Enable=1)
90     NVIC_EnableIRQ(TIMER0_IRQn); //Habilitación de Interrupción por TIMER0
91     return;
92 }
93
```

Ejemplo 2: Interrupción por CAPTURE del TMR0

```
94 void TIMER0_IRQHandler(void)
95 {
96     if(TIM_GetIntCaptureStatus(LPC_TIM0, TIM_CR0_INT)){           //Consulta Interrupción por CR0 (CAPTURE0 del Timer0)
97         //if(LPC_TIM0->IR & (1<<4+0)){
98         //Ejecución de la ISR
99         currCapValue = TIM_GetCaptureValue(LPC_TIM0,TIM_COUNTER_INCAP0);           //Valor del Registro de Captura CAP0
100         //value = LPC_TIM0->CR0;
101
102         if(capFlag == 0)
103         {
104             //Guarda primera captura
105             lastCapValue = currCapValue;
106             capFlag = 1;
107         } else {
108             //Calcula diferencia de ticks entre dos capturas
109             uint32_t diffTicks = (currCapValue - lastCapValue); //ticks transcurridos entre los dos eventos
110             //Calcula tiempo en segundos: PCLK = CCLK/4 = 25[MHz], PR=0 -> tick = 1/25[MHz] = 40[ns]
111             float tickTime = 1.0f / 25000000.0f; // 40[ns]
112             float diffTime = diffTicks * tickTime; // segundos
113             if(diffTime >= 1.0f) // Si pasó 1 segundo o más ...
114             {
115                 GPIO_SetValue(PORT_0,PIN_22);
116             }
117             //Actualiza para la próxima medición
118             lastCapValue = currCapValue;
119         }
120     //}
121 }
122
123 TIM_ClearIntCapturePending(LPC_TIM0, TIM_MR0_INT);           //Limpia Bandera de Interrupción por TIMER0
124 //LPC_TIM0->IR |= (1<<4+0);
125 return;
126 }
```